1. **To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

Task -1: Create Project Structure as follows:

```
OSSP/
│
├── .git/                # Git repository (created automatically)
├── .gitignore           # Files/folders Git should ignore
├── README.md            # Project description and instructions
├── LICENSE              # Project license (optional)
├── Makefile             # Build instructions
│
├── src/                 # Source code
│   ├── main.c
│   ├── shell.c
│   ├── parser.c
│   ├── executor.c
│   └── builtin.c
│
├── include/             # Header files
│   ├── shell.h
│   ├── parser.h
│   ├── executor.h
│   └── builtin.h
│
├── obj/                 # Object files (.o) (generated)
│
├── bin/                 # Executable file (generated)
│   └── my_shell
│
├── docs/                # Documentation
│   ├── design.md
│   └── report.pdf
│
├── tests/               # Test programs
│   ├── test_parser.c
│   └── test_executor.c
│
├── scripts/             # Helper scripts
│   └── run.sh
│
└── [illegible]          # [illegible]
```

└── assets/ # Images/screenshots
└── architecture.png

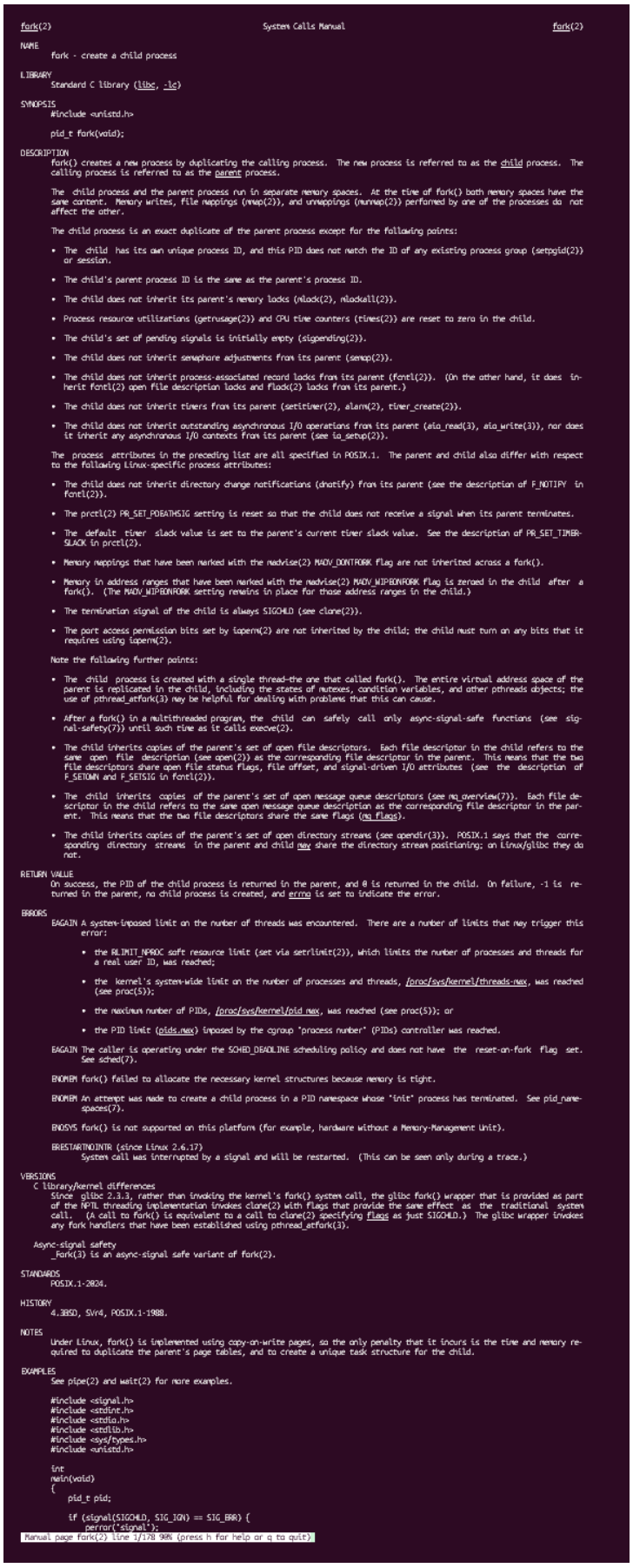
Note: For creating this structure use mkdir and touch commands.

```
speed@DESKTOP-P4G181S:~$ # 1. Create main directory and enter it
mkdir -p OSSP && cd OSSP

# 2. Initialize Git repository
git init

# 3. Create all subdirectories
mkdir -p src include obj bin docs tests scripts assets

# 4. Create root-level files
touch .gitignore README.md LICENSE Makefile

# 5. Create source files in src/
touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c

# 6. Create header files in include/
touch include/shell.h include/parser.h include/executor.h include/builtin.h

# 7. Create documentation, test, script, and asset files
touch docs/design.md docs/report.pdf
touch tests/test_parser.c tests/test_executor.c
touch scripts/run.sh
touch assets/architecture.png

# 8. (Optional) Verify the directory tree structure
tree -a
Reinitialized existing Git repository in /home/speed/OSSP/.git/
```


.
├── .git
│   ├── HEAD
│   ├── config
│   ├── description
│   ├── hooks
│   │   ├── applypatch-msg.sample
│   │   ├── commit-msg.sample
│   │   ├── fsmonitor-watchman.sample
│   │   ├── post-update.sample
│   │   ├── pre-applypatch.sample
│   │   ├── pre-commit.sample
│   │   ├── pre-merge-commit.sample
│   │   ├── pre-push.sample
│   │   ├── pre-rebase.sample
│   │   ├── pre-receive.sample
│   │   ├── prepare-commit-msg.sample
│   │   ├── push-to-checkout.sample
│   │   ├── sendemail-validate.sample
│   │   └── update.sample
│   ├── info
│   │   └── exclude
│   ├── objects
│   │   ├── info
│   │   └── pack
│   └── refs
│   ├── heads
│   └── tags
├── .gitignore
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── shell
├── shell.c
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

1. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

Task-1: Using a manual command understand the fork() and exec() system calls



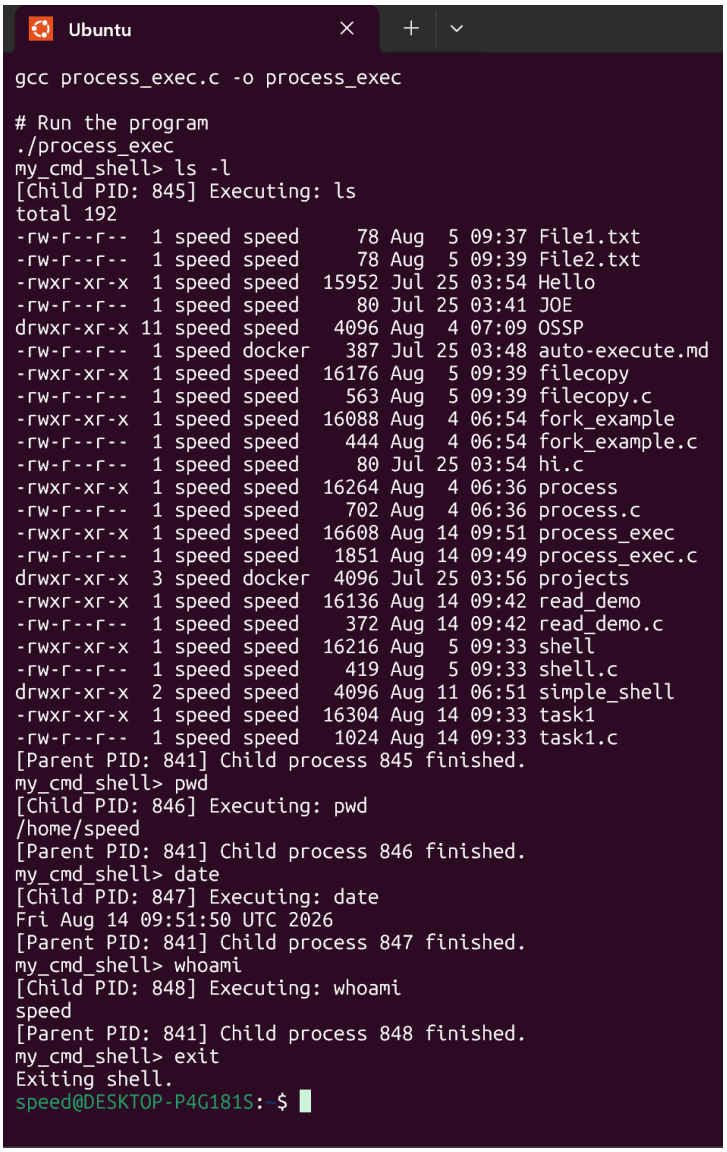
Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Example:

```
#include <stdio.h>
```

```c
#include <unistd.h>
int main() {
    pid_t pid;
    pid = fork();   // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```
 GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        // Display prompt
        printf("my_cmd_shell> ");
        fflush(stdout);

        // Read command from user
        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nExiting...\n");
            break;
        }

        // Remove trailing newline character
        input[strcspn(input, "\n")] = '\0';

        // Skip empty input
        if (strlen(input) == 0) {
            continue;
        }

        // Handle exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell.\n");
            break;
        }

        // Parse command into arguments (tokenize by space)
        int i = 0;
        char *token = strtok(input, " ");
        while (token != NULL && i < MAX_ARGS - 1) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }
        args[i] = NULL; // Array must be NULL-terminated for execvp

        // Create child process
        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
        }
        else if (pid == 0) {
            // Child Process: Execute the command
            printf("[Child PID: %d] Executing: %s\n", getpid(), args[0]);
            if (execvp(args[0], args) < 0) {
                perror("Command execution failed");
                exit(EXIT_FAILURE);
            }
        }
        else {
            // Parent Process: Wait for child process to terminate
            int status;
            waitpid(pid, &status, 0);
            printf("[Parent PID: %d] Child process %d finished.\n", getpid(), pid);
        }
    }

    return 0;
}
```


Ubuntu
gcc process_exec.c -o process_exec
Run the program
./process_exec
my_cmd_shell> ls -l
[Child PID: 845] Executing: ls
total 192
-rw-r--r-- 1 speed speed   78 Aug 5 09:37 File1.txt
-rw-r--r-- 1 speed speed   78 Aug 5 09:39 File2.txt
-rwxr-xr-x 1 speed speed 15952 Jul 25 03:54 Hello
-rw-r--r-- 1 speed speed   80 Jul 25 03:41 JOE
drwxr-xr-x 11 speed speed   4096 Aug 4 07:09 OSSP
-rw-r--r-- 1 speed docker   387 Jul 25 03:48 auto-execute.md
-rwxr-xr-x 1 speed speed 16176 Aug 5 09:39 filecopy
-rw-r--r-- 1 speed speed   563 Aug 5 09:39 filecopy.c
-rwxr-xr-x 1 speed speed 16088 Aug 4 06:54 fork_example
-rw-r--r-- 1 speed speed   444 Aug 4 06:54 fork_example.c
-rw-r--r-- 1 speed speed   80 Jul 25 03:54 hi.c
-rwxr-xr-x 1 speed speed 16264 Aug 4 06:36 process
-rw-r--r-- 1 speed speed   702 Aug 4 06:36 process.c
-rwxr-xr-x 1 speed speed 16608 Aug 14 09:51 process_exec
-rw-r--r-- 1 speed speed   1851 Aug 14 09:49 process_exec.c
drwxr-xr-x 3 speed docker 4096 Jul 25 03:56 projects
-rwxr-xr-x 1 speed speed 16136 Aug 14 09:42 read_demo
-rw-r--r-- 1 speed speed   372 Aug 14 09:42 read_demo.c
-rwxr-xr-x 1 speed speed 16216 Aug 5 09:33 shell
-rw-r--r-- 1 speed speed   419 Aug 5 09:33 shell.c
drwxr-xr-x 2 speed speed   4096 Aug 11 06:51 simple_shell
-rwxr-xr-x 1 speed speed 16304 Aug 14 09:33 task1
-rw-r--r-- 1 speed speed   1024 Aug 14 09:33 task1.c
[Parent PID: 841] Child process 845 finished.
my_cmd_shell> pwd
[Child PID: 846] Executing: pwd
/home/speed
[Parent PID: 841] Child process 846 finished.
my_cmd_shell> date
[Child PID: 847] Executing: date
Fri Aug 14 09:51:50 UTC 2026
[Parent PID: 841] Child process 847 finished.
my_cmd_shell> whoami
[Child PID: 848] Executing: whoami
speed
[Parent PID: 841] Child process 848 finished.
my_cmd_shell> exit
Exiting shell.
speed@DESKTOP-P4G181S:~$